DETAILED PROJECT REPORT

cvfai: Deep Edge Detection with Holistically-Nested Networks

Background, method, experimental evidence, implementation with hed-tensor, evaluation protocol and roadmap

Prepared October 9, 2026 · Core reference: Xie & Tu, arXiv:1504.06375 (ICCV 2015) · Tooling: `hed-tensor` on PyPI

Contents

1. Executive summary
2. Background and motivation
3. Related approaches
4. The HED method
5. Network architecture
6. Training recipe
7. Experimental results
8. Ablations and insights
9. Implementation with hed-tensor
10. Project pipeline
11. Evaluation protocol
12. Limitations and risks
13. Future work
14. Glossary
15. References

Scope note. Sections on the method and results are drawn from the HED paper and the `hed-tensor` package page. Anything specific to your own cvfai work (dataset, your measured results, application, the reference video) was not available to me and is marked *[to fill in]*.

1 Executive summary

cvfai is a computer-vision project centred on **Holistically-Nested Edge Detection (HED)**, a convolutional approach that turns an input image into an edge map in one forward pass. HED combines two ideas: *holistic* image-to-image prediction (as in fully convolutional networks) and *nested, deeply supervised* multi-scale learning (side outputs on several network stages that are trained directly and then fused).

When published, HED set the best reported scores on two standard benchmarks while running orders of magnitude faster than earlier CNN edge detectors. The project uses

`hed-tensor`, a TensorFlow/Keras port that ships pretrained weights, to make HED usable in a few lines of Python.

.782	.790	.746	0.4 s
BSDS500 ODS (paper v1)	BSDS500 ODS (retrained, appendix)	NYUDv2 ODS	per 320×480 image, GPU

2 Background and motivation

Edge and object-boundary detection underpins many vision tasks: segmentation, object recognition, tracking, medical imaging, 3D reconstruction, and more recently conditioning signals for image generation. Human annotators disagree somewhat on where to draw boundaries but are broadly consistent; the paper cites an inter-annotator F-score of about 0.80 on BSDS500, which serves as a practical ceiling.

Classical detectors (Sobel, zero-crossing, Canny) respond to local gradients at a chosen scale. Changing the scale parameter moves and fragments edges, and there is no notion of which contours are semantically important. Learned detectors address this, but the first deep approaches worked on small patches, which made prediction slow (seconds to hours per image) and gave hidden layers no direct training signal.

The research questions HED answers are therefore: (1) can edges be trained and predicted holistically, on whole images? and (2) can multi-scale, multi-level features be learned inside a single network instead of through hand-built pyramids?

3 Related approaches

The paper groups prior work into early hand-designed filters, information-theoretic methods with engineered features (e.g. Pb, gPb), learning methods that still rely on human-designed features (e.g. Sketch Tokens, Structured Edges), and patch-based CNNs (N^4-Fields, DeepContour, DeepEdge, CSCNN). It also distinguishes four ways of obtaining multi-scale behaviour in deep nets, and positions HED as a fifth:

Configuration	Idea	Drawback for edge detection

Multi-stream	Parallel streams with different receptive fields, features concatenated	Redundant computation; one loss, one output
Skip-layer	Single stream with links from several depths into a shared output layer	Single output loss; no explicit per-scale prediction
One model, multiple input scales	Run (tied) network on resized inputs and combine	Slows inference, especially with deep models
Independent networks	Train separate nets with different depths and losses	Multiplies training resources
Holistically-nested (HED)	**One stream, several side outputs, each supervised, plus a learned fusion**	**Little redundancy; per-scale outputs come for free**

4 The HED method

4.1 Problem formulation

Each training sample is a pair of an image X and a binary ground-truth edge map Y of the same size. The network has M side-output layers ($M = 5$), each with its own classifier weights, on top of the shared network parameters W.

4.2 Class-balanced loss

Around 90% of pixels in a natural image are non-edge, so an unweighted loss lets the network get a low error by predicting "no edge" almost everywhere. HED balances the two classes per image with a weight β equal to the fraction of non-edge pixels:

$$\ell_{side}^{(m)} = -\beta \cdot \Sigma_{j \in Y+} \log \Pr(y_j=1) - (1-\beta) \cdot \Sigma_{j \in Y-} \log \Pr(y_j=0), \quad \beta = |Y-| \,/\, |Y|$$

$\Pr(y_j=1)$ is a sigmoid of the activation at pixel j. Because β is computed per image, no manual sampling ratio is needed.

4.3 Deep supervision and weighted fusion

The total side-output objective sums the per-layer losses with weights α_m (all set to 1). A weighted-fusion layer then learns coefficients h_m to combine the side activations into one prediction, trained with a cross-entropy loss. Everything is optimised jointly:

$$L_{side} = \Sigma_{m=1..M}\ \alpha_m\ \ell_{side}^{(m)} \quad \hat{Y}_{fuse} = \sigma(\ \Sigma_m\ h_m\ \hat{A}_{side}^{(m)}\ ) \quad (W, w, h)^* = \text{argmin}\ (\ L_{side} + L$$

4.4 Inference

At test time the network emits the fused map plus all five side maps in one pass. The final output can be the fusion map alone or an average of several maps. The paper's "late merging" averages the fusion output with the side outputs to raise average precision, at no extra cost because the side outputs already exist.

---

5 Network architecture

HED adapts **VGG-16** with two changes. First, a side output is attached to the last convolution of each of the five stages (conv1_2, conv2_2, conv3_3, conv4_3, conv5_3). Second, the fifth pooling layer and all fully connected layers are removed. Dropping the stride-32 stage avoids very coarse outputs that blur when interpolated, and removing the dense layers cuts memory and time. The result has five stages with strides 1, 2, 4, 8 and 16.

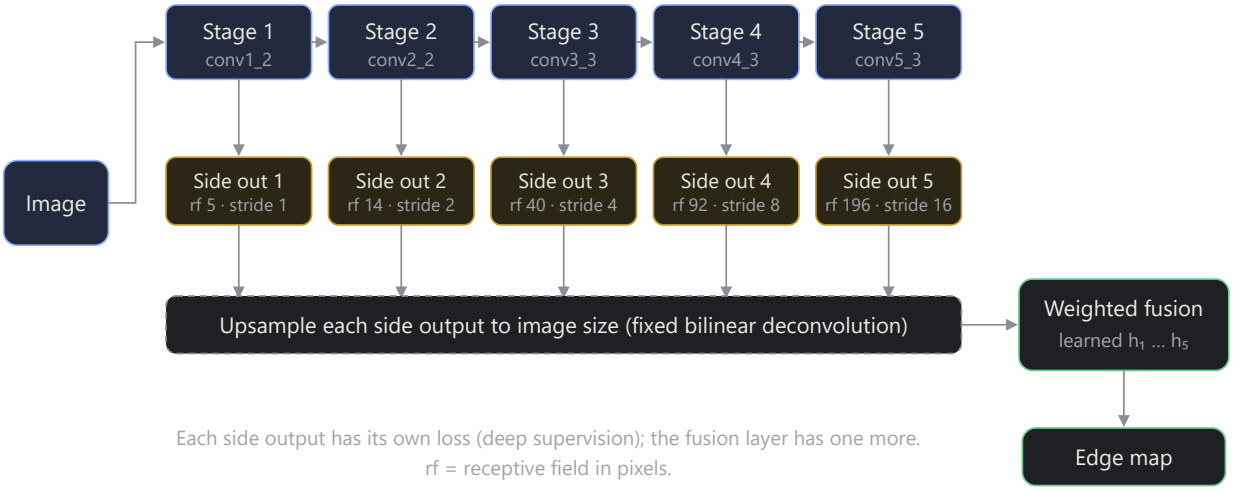


Figure 1. HED architecture: trimmed VGG-16 with five supervised side outputs and a learned fusion layer.

Side output	Attached to	Receptive field	Stride	Behaviour
1	conv1_2	5	1	Fine local edges, many weak/texture

1	conv1_2	5	1	Fine local edges, many weak/texture responses
2	conv2_2	14	2	Local contours
3	conv3_3	40	4	Mid-level boundaries
4	conv4_3	92	8	Object-level boundaries
5	conv5_3	196	16	Coarse, global structure

6 Training recipe (original work)

The original system was built in Caffe, building on public FCN and deeply-supervised-net implementations, and fine-tuned from ImageNet-pretrained VGG-16.

Setting	Value
Mini-batch size	10
Learning rate	1e-6, divided by 10 after 5,000 iterations
Iterations	10,000
Momentum / weight decay	0.9 / 0.0002
Side-output loss weights α_m	1 each
Fusion weights initialisation	1/5 each
Training image size (v1)	Resized to 400 × 400
Hardware / time	Single NVIDIA K40 GPU, about 7 hours

Notable training details

- **Consensus sampling.** A pixel counts as an edge only if at least three annotators marked it. Without this, noisy weak edges caused exploding gradients in the high-level side outputs, whose coarse predictions cannot represent them.
- **Augmentation.** Rotation to 16 angles (cropping the largest valid rectangle) plus flips, making the training set 32× larger. The appendix retrain also added 50/100/150% rescaling and trained on full-resolution images.

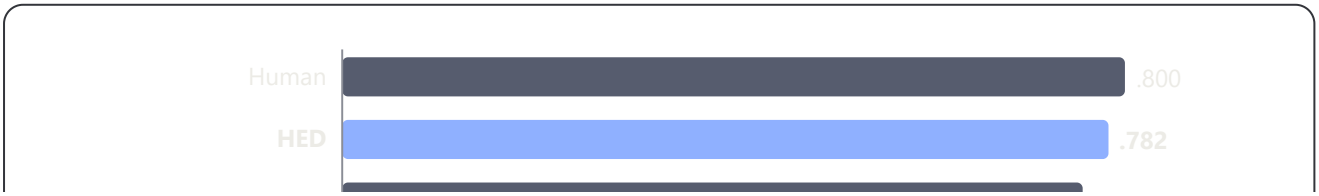
- **Fixed bilinear upsampling.** Learned deconvolutions gave no visible gain over fixed bilinear interpolation.
- **Max pooling.** Swapping to average pooling lowered ODS to .741.
- **Extra non-linearity.** Adding a 50-filter ReLU layer before each side output hurt performance.
- **Ensemble testing.** Averaging predictions over rotated/flipped inputs gave no improvement, so inference stays single-pass.

7 Experimental results

7.1 BSDS500

BSDS500 contains 200 training, 100 validation and 200 test images with multiple human-drawn contours each. Scores use three measures: ODS (one threshold for the whole dataset), OIS (best threshold per image) and AP (average precision), after standard non-maximum suppression and thinning.

Method	ODS	OIS	AP	Speed (FPS)
Human	.80	.80	–	–
Canny	.600	.640	.580	15
gPb-owt-ucm	.726	.757	.696	1/240
Sketch Tokens	.727	.746	.780	1
Structured Edges (SE-Var)	.746	.767	.803	2.5
DeepEdge	.753	.772	.807	1/103 (GPU)
DeepContour	.756	.773	.797	1/30 (GPU)
HED (paper v1)	**.782**	**.804**	**.833**	**2.5 (GPU), 1/12 (CPU)**



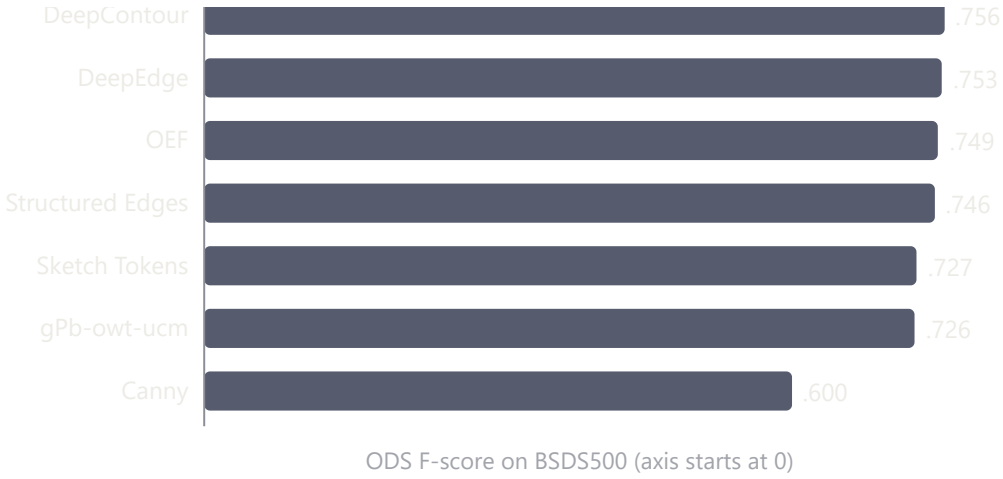


Figure 2. ODS F-score on BSDS500 for selected methods (values from the original paper).

7.2 Retrained model (paper appendix)

After the conference submission the authors retrained with extra scale augmentation and full-resolution images:

Variant	ODS	OIS	AP
Fusion output, deep supervision	**.790**	**.808**	**.811**
Fusion output, no deep supervision	.785	.801	.730
HED with late merging	.788	.808	.840

The released pretrained Caffe model corresponds to this stronger setting (ODS .790, OIS .808).

7.3 More training data

Moving 100 of the 200 test images into training (averaged over five random splits, evaluated on the remaining 100) raised ODS from .782 to about .797 (±.003). The authors take this as evidence that data, not model capacity, is the main bottleneck.

7.4 NYU Depth v2

NYUDv2 has 1,449 RGB-D images (381 train, 414 validation, 654 test). Depth was encoded as HHA channels (horizontal disparity, height above ground, angle to gravity); one HED was trained on RGB, another on HHA, and their predictions averaged. For evaluation the matching tolerance was relaxed from .0075 to .011, following earlier work, and the Average 2–4 side outputs were used instead of the fusion layer.

Method	ODS	OIS	AP	FPS
gPb+NG	.687	.716	.629	1/375
SE	.685	.699	.679	5
SE+NG+	.710	.723	.738	1/15
HED-RGB	.720	.734	.734	2.5 (GPU)
HED-HHA	.682	.695	.702	2.5 (GPU)
HED-RGB-HHA	**.746**	**.761**	**.786**	**1 (GPU)**

An interesting takeaway: even for a deep network, carefully engineered depth features (HHA) worked better than feeding raw depth, particularly with limited training data.

7.5 Speed

A 320×480 image takes about 400 ms on a GPU including interface overhead, and about 12 seconds on a CPU. Competing CNN methods needed from several seconds to well over a minute per image on GPU.

---

8 Ablations and insights

8.1 Why deep supervision matters

Architecture	ODS	OIS	AP
FCN-8s (edge loss)	.697	.715	.673
FCN-2s (extra low-level links)	.738	.756	.717
Fusion only, no deep supervision	.771	.785	.738
Fusion with deep supervision (HED)	**.782**	**.802**	**.787**

Without deep supervision the F-score drops only slightly, but average precision falls sharply and the side outputs lose their orderly coarse-to-fine character: the network drifts toward large structural edges and loses fine boundaries. With supervision, side

outputs progress cleanly from local to global.

8.2 What each side output contributes

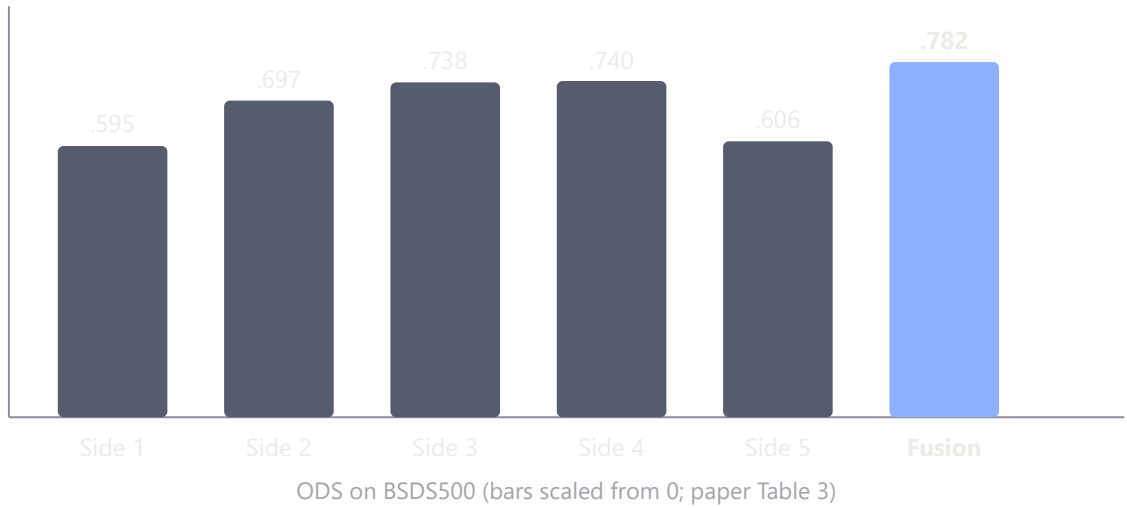


Figure 3. Individual side outputs peak in the middle stages; the learned fusion beats every single output.

- Middle side outputs (3 and 4) are individually strongest; side 1 is noisy (too many texture edges) and side 5 is too coarse.
- Even the weak outputs help in combination. Averaging sides 1–4 gives ODS .760, and adding side 5 raises it to .774.
- Averaging side outputs favours average precision (AP .822 for sides 1–5), whereas the fusion layer favours F-score. Merging the two gives the best overall numbers: ODS .782, OIS .804, AP .833.

8.3 Behaviour of deep versus classical detectors

Deep detectors lose more precision than classical ones in the high-recall regime because they learn to favour global object boundaries and skip weak edges. HED holds up better than other CNN methods there, because deep supervision keeps the low-level responses in play.

---

9 Implementation with `hed-tensor`

`hed-tensor` is a community TensorFlow/Keras implementation of HED with **pretrained weights bundled in the package**. PyPI lists two releases, 1.0.0 and 1.0.1, both published on October 20, 2025. The project positions it as being as easy to

1.0.1, both published on October 29, 2023. The project positions it as being as easy to use as Keras' built-in VGG16 or ResNet.

Feature	Detail
Installation	`pip install hed-tensor`
Main objects	`HED` (model), `detect_edges`, `batch_detect_edges`
Input types	File path, PIL image, or NumPy array
Weights	Bundled and loaded automatically; `HED(weights='path.h5')` loads a custom checkpoint; `HED(weights=None)` gives untrained weights for training from scratch
Framework	TensorFlow 2.x

Minimal usage

```
from hed_tensor import HED, detect_edges
from PIL import Image

model = HED()                                    # pretrained weights load automatically
edges = detect_edges('image.jpg', model=model)  # edge map as an array
Image.fromarray(edges).save('edges.png')
```

Batch usage

```
from hed_tensor import HED, batch_detect_edges

model = HED()
# see the package page for the exact argument names and return type
results = batch_detect_edges(list_of_images, model=model)
```

Fine-tuning from a checkpoint (outline)

```
model = HED(weights='path/to/your/checkpoint.h5')   # resume or specialise
model = HED(weights=None)                           # train from scratch
```

Verify before claiming. The package page does not state which dataset the bundled weights were trained on, whether they were converted from the original Caffe model, or whether side outputs are exposed. Before reporting accuracy numbers, test the package yourself (see section

11) and cite the original paper, as the package itself requests.

---

10 Project pipeline

1. **Data intake.** Load images from files or arrays. *[to fill in: source, count, resolution]*
2. **Pre-processing.** Ensure 3-channel RGB; keep original aspect ratio where possible (HED was designed for original-size inputs at test time).
3. **Inference.** `detect_edges` for single images, `batch_detect_edges` for folders.
4. **Post-processing.** HED maps are soft and several pixels wide. For crisp contours, threshold and thin the map. Illustrative code (not part of the package; assumes an 8-bit single-channel output):

```
import numpy as np
from skimage.morphology import skeletonize

binary = edges > 128                # choose threshold on a validation set
thin   = skeletonize(binary)        # one-pixel-wide contours
Image.fromarray((thin * 255).astype(np.uint8)).save('edges_thin.png')
```

5. **Output and downstream use.** Save edge maps, overlay on the image, or pass to the next stage. *[to fill in: application]*

---

11 Evaluation protocol

11.1 Qualitative

- Panels of original image, Canny (several σ), Sobel and HED, using varied scenes (clutter, texture, low light, indoor, outdoor).
- Highlight the paper's point: Canny contours shift and fragment across scales, while HED stays consistent.

11.2 Quantitative

- Use BSDS500 test images with ground truth. Apply NMS and thinning, then compute ODS, OIS and AP with the standard BSDS benchmark code.

- Compare against your own numbers for Canny and Sobel baselines under the same protocol.
- Expect results near, but not necessarily identical to, the paper's .78–.79, since this is a different framework and weight conversion.

11.3 Efficiency

- Latency per image on CPU and GPU, and throughput for batch inference at several batch sizes.
- Peak memory at the image sizes you actually use.

11.4 Your results

Experiment	Setting	Result
Qualitative comparison	*[to fill in]*	*[to fill in]*
ODS / OIS / AP	*[to fill in]*	*[to fill in]*
Latency (CPU / GPU)	*[to fill in]*	*[to fill in]*

---

12 Limitations and risks

- **Thick, soft output.** Needs thinning for precise localisation; evaluation on BSDS requires NMS.
- **No explicit context modelling.** The authors themselves note HED does not enforce constraints between neighbouring pixel labels.
- **Weak edges are suppressed.** Consensus-based labels and deep features deliberately favour strong, object-level boundaries, which may be wrong for tasks that need fine texture edges.
- **Data-limited.** BSDS500 is small; the paper shows more data helps, and domain shift (medical, aerial, document images) can degrade results.
- **Dated backbone.** VGG-16 is heavy by today's standards, and newer edge and contour detectors have reported higher scores since 2015.
- **Package maturity.** `hed-tensor` is a recent community release with two versions; weight provenance should be verified before publishing claims.

13 Future work

- Fine-tune on a domain dataset using a custom checkpoint, adding data augmentation as in the paper (rotations, flips, scaling).
- Experiment with the late-merging idea: combine the fusion output with averaged side outputs if the package exposes them.
- Benchmark against newer detectors and lightweight backbones for mobile or real-time use.
- Build a small upload-and-detect demo (notebook or web app).
- Use edge maps as conditioning input for downstream segmentation or generative models.

14 Glossary

Term	Meaning
ODS	Optimal Dataset Scale: F-score using one fixed threshold across the whole dataset
OIS	Optimal Image Scale: F-score using the best threshold for each image
AP	Average precision over the precision–recall curve
NMS	Non-maximum suppression: thinning thick responses to local maxima
Deep supervision	Attaching losses directly to intermediate layers, not only the output
Side output	A prediction branch attached to an intermediate network stage
Receptive field	The image region that influences one unit of a layer
HHA	Depth encoding with horizontal disparity, height above ground and angle with gravity
FCN	Fully convolutional network: dense, image-to-image prediction